



UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
(Universidad del Perú, DECANA DE AMÉRICA)

FACULTAD DE INGENIERIA DE SISTEMAS E INFORMATICA
ESCUELA PROFESIONAL DE INGENIERÍA DE SOFTWARE

SILABO

1. INFORMACIÓN GENERAL

1.1	Nombre de la asignatura	: TENDENCIAS DE ARQUITECTURA DE SOFTWARE – Plan 2018
1.2	Código de la asignatura	: 202W1005
1.3	Tipo de asignatura	: Obligatorio
1.4	Área de Estudios	: Complementario
1.5	Número de Semanas	: 16
1.6	Horas semanales	: 4 (Teoría: 2, Práctica: 0, Laboratorio 2)
1.7	Semestre Académico	: 2026-II
1.8	Ciclo	: X
1.9	Créditos	: 3
1.10	Modalidad	: Presencial
1.11	Pre-requisitos	: 202W0907 – TALLER DE CONSTRUCCIÓN DE SOFTWARE MÓVIL
1.12	Docente(es)	: CORDERO SÁNCHEZ, HUGO R. (hcorderos@unmsm.edu.pe) (coord.) QUIÑONES NIETO, YAMIL ALEXANDER yquinonesn@unmsm.edu.pe

2. SUMILLA

Esta asignatura corresponde al área de Estudios de Especialidad, es de naturaleza teórico y práctico; tiene el propósito de introducir a los estudiantes en el conocimiento y aplicación de las nuevas arquitecturas de software para lograr la competencia: “Desarrolla y mantiene soluciones de software utilizado metodologías, métodos, técnicas y herramientas de software basado en estándares internacionales de calidad, con actitud crítica, creativa y con trabajo en equipo”. Los principales contenidos son: Definiciones y Contexto de la arquitectura de Software, Patrones y Frameworks, Análisis de decisiones y Resolución, Metas del negocio, Atributos de Calidad, Arquitectura de Referencia, Desarrollo de una Arquitectura, Arquitecturas de modelos de software como servicio (SaaS).

3. COMPETENCIAS DEL PERFIL DE EGRESO A LA QUE CONTRIBUYE LA ASIGNATURA

Código	Descripción	Tipo	Nivel
CT8.3	Integra e implementa soluciones de diferentes tipos de software; utilizando metodologías, métodos, técnicas y herramientas de software basado en estándares de calidad	Técnica	Avanzado

4. LOGROS DE APRENDIZAJE (Competencias de la asignatura)

CT8.3 RA1: Diseñar soluciones de software que utilicen métodos y arquitecturas en tendencia, aplicando los conocimientos en diseño y arquitectura de software.

CT8.3 RA2: Desarrollar aplicaciones de software con arquitecturas y tecnologías en nube, utilizando tendencias de arquitecturas de software.

CT8.3 RA3: Integrar aplicaciones de software utilizando tecnologías en tendencia, a partir del desarrollo de pruebas de concepto y demos.

CT8.3 RA4: Identificar y proponer la utilización de plataformas de software en tendencia para la solución de problemas reales y proyectos.

5. CAPACIDADES (Logros por unidad)

Unidad I:

- Conocer las tendencias en Arquitectura de Software y Patrones, planificar el proyecto del curso y diseñar la solución utilizando las arquitecturas seleccionadas: CT8.3 - RA1

Unidad II:

- Entender las tecnologías de nube y su relación con las Arquitecturas en tendencia para desarrollar aplicaciones, demostrando trabajo en equipo y capacidad de análisis: CT8.3 - RA1

Unidad III:

- Identificar, proponer e integrar otras tecnologías y plataformas en tendencia para la Arquitectura de software, a través de su aplicación en problemas reales, demostrando pensamiento crítico: CT8.3 - RA3, CT8.3 - RA4

6. PROGRAMACIÓN DE CONTENIDOS

Unidad I: INTRODUCCIÓN A LA ARQUITECTURA DE SOFTWARE Y TENDENCIAS				
Logros por unidad: • Conocer las tendencias en Arquitectura de Software y Patrones, planificar el proyecto del curso y diseñar la solución utilizando las arquitecturas seleccionadas. CT8.3 - RA1				
Sem	Contenido	Actividades	Recursos	Estrategias

1	ARQUITECTURA DE SOFTWARE Y TENDENCIAS Definición de arquitectura de software. Importancia. Rol del arquitecto. Evolución de la arquitectura de software. Tendencias generales en desarrollo de software. Tendencias en computación en la nube. Panorama actual de tendencias en arquitectura de software. Tendencias en Arquitectura de Software.	• Presentación del curso • Introducción personal. • Explicación del proyecto del curso. • Formación de Equipos de Trabajo. • Seleccionar tendencias para su proyecto del curso. • Analizar caso de planteamiento de arquitectura	• Silabo • Diapositivas de clase • Lineamientos del proyecto • Software: Jira	• Aprendizaje activo: Los estudiantes aprenden haciendo. • Aprendizaje colaborativo: Los estudiantes trabajan en equipo
2	DRIVERS ARQUITECTÓNICOS Y MÉTODOS DE DISEÑO DE LA ARQUITECTURA Atributos de Calidad. Principales atributos. Escenarios de atributos. Tácticas para Atributos. Ciclo de desarrollo de la Arquitectura. Drivers Arquitectónicos y métodos de identificación de drivers. Métodos de Diseño de la Arquitectura.	• Desarrollo de escenarios de atributos de calidad • Identificación de drivers en casos prácticos • Aplicación de método de diseño para la arquitectura • Videos de identificación de drivers y métodos de diseño	• Diapositivas de clase • Casos prácticos para clase. • Software: Draw.io	• Aprendizaje activo: Los estudiantes aprenden haciendo. • Aprendizaje colaborativo: Los estudiantes trabajan en equipo
3	DOCUMENTACIÓN DE LA ARQUITECTURA Introducción a la documentación. Las Vistas para la arquitectura y su aplicación. Modelo de Vistas: modelo 4+1, modelo de Siemens, modelo de estructuras del SEI y el modelo C4. Ejemplos de vistas. Notaciones para representar la arquitectura. Registros de Decisiones de Arquitectura (ADR).	• Análisis de casos para la representación de las vistas de la arquitectura. • Desarrollo de un modelo C4 para un sistema. • Diagramación en herramienta de C4	• Diapositivas de clase • Casos prácticos para el desarrollo de Vistas • Caso práctico para aplicar el modelo C4 • Software: Structurizr y IcePanel	• Aprendizaje activo: Los estudiantes aprenden haciendo. • Aprendizaje colaborativo: Los estudiantes trabajan en equipo

Unidad II: ESTILOS Y PATRONES ARQUITECTÓNICOS PARA SISTEMAS MODERNOS

Logros por unidad:

- Entender las tecnologías de nube y su relación con las Arquitecturas en tendencia para desarrollar aplicaciones, demostrando trabajo en equipo y capacidad de análisis.
CT8.3 - RA1

Sem	Contenido	Actividades	Recursos	Estrategias
-----	-----------	-------------	----------	-------------

4	ARQUITECTURA DESACOPADAS Y CENTRADAS EN EL DOMINIO Principios de diseño SOLID orientado a la Arquitectura. Los 5 principios. El Diseño Dirigido por Dominio (DDD). Arquitectura Hexagonal. Ventajas y desventajas. Objetivos que persigue. Componentes de la Arquitectura Hexagonal. Regla de dependencias. Arquitectura Cebolla. Arquitectura Limpia. Ventajas y desventajas.	• Desarrollo de ejemplos con principios SOLID • Identificación de casos donde aplicar DDD • Desarrollo de ejemplo con DDD • Desarrollo de ejemplo con Arquitectura Hexagonal, Arquitectura Cebolla y Arquitectura Limpia	• Diapositivas de clase • Casos prácticos para el desarrollo de SOLID • Casos prácticos para Arquitectura Hexagonal, Arquitectura Cebolla y Arquitectura Limpia.	• Aprendizaje activo: Los estudiantes aprenden haciendo. • Aprendizaje colaborativo: Los estudiantes trabajan en equipo
5	ARQUITECTURA CLOUD NATIVE Y EVOLUCIÓN DEL MONOLITO Contenedores. Orquestados de contenedores. Microservicios. Diferencias con Arq. Monolito. Principales características de los Microservicios. Aplicaciones nativo en nube. API Gateway. Monolito modular como estrategia evolutiva	• Desarrollo de ejemplo con Microservicios • Desarrollo de ejemplo con API Gateway	• Diapositivas de clase • Casos para aplicar Microservicios	• Aprendizaje activo: Los estudiantes aprenden haciendo. • Aprendizaje colaborativo: Los estudiantes trabajan en equipo
6	PATRONES DE INTEGRACIÓN Y COMUNICACIÓN DISTRIBUIDA Patrón de Segregación de responsabilidades de consultas de comandos (CQRS). Arquitecturas dirigidas por eventos. Recomendaciones de implementación. Patrón Saga. Versiones Orquestador y Coreografía. gRPC. GraphQL. Comparación con REST y gRPC. Ejemplos y mejores prácticas con GraphQL. Microservicios con GraphQL.	• Desarrollo de ejemplo con CQRS • Desarrollo de ejemplo con Event-Driven • Desarrollo de ejemplo con Saga • Desarrollo de ejemplo con GraphQL • Desarrollo de ejemplo con gRPC	• Diapositivas de clase • Casos para aplicar patrones con Microservicios • Diapositivas de clase • Casos para aplicar GraphQL con Microservicios.	• Aprendizaje activo: Los estudiantes aprenden haciendo. • Aprendizaje colaborativo: Los estudiantes trabajan en equipo
7	SERVERLESS Y MONITOREO, OBSERVABILIDAD ARQUITECTÓNICA Arquitectura sin servidor. FaaS y BaaS. Casos de uso de Serverless. Diferencias frente a microservicios y estilos tradicionales. Monitoreo en arquitecturas distribuidas. Observabilidad: logs, métricas y trazas. Introducción a SLI, SLO y alertamiento. Relación entre Observabilidad, calidad y mantenibilidad.	• Desarrollo de ejemplo con Funciones como servicio. • Desarrollo de ejemplo invocando a servicio externo • Casos prácticos de Monitoreo y Observabilidad.	• Diapositivas de clase • Casos para aplicar Sin Servidor. • Herramientas de Monitoreo y Observabilidad	• Aprendizaje activo: Los estudiantes aprenden haciendo. • Aprendizaje colaborativo: Los estudiantes trabajan en equipo
8	EXPOSICIÓN PARCIAL DEL PROYECTO			

Unidad III: TENDENCIAS EMERGENTES EN ARQUITECTURA DE SOFTWARE				
Logros por unidad: Identificar, proponer e integrar otras tecnologías y plataformas en tendencia para la Arquitectura de software, a través de su aplicación en problemas reales, demostrando pensamiento crítico: CT8.3 - RA3, CT8.3 - RA4				
Sem	Contenido	Actividades	Recursos	Estrategias
9	IA GENERATIVA Y TRANSFORMACIÓN DE LA ARQUITECTURA IA generativa. Impacto de la IA generativa en la Ingeniería de Software. Desarrollo asistido por IA. Ingeniería de Software IA Nativa. Cambios en el rol del arquitecto. Riesgos de opacidad, dependencia y deuda técnica. Arquitectura para soluciones construidas con apoyo de IA.	- Ejemplos de aplicaciones con IA generativa. - Discusión del rol del arquitecto. - Práctica de desarrollo de software soportado con IA	- Diapositivas de clase - Ejemplo práctico del desarrollo de una aplicación con IA.	- Aprendizaje activo: Los estudiantes aprenden haciendo. - Aprendizaje colaborativo: Los estudiantes trabajan en equipo
10	ARQUITECTURAS PARA SISTEMAS CON LLM Sistemas con LLM. Integración de LLM en aplicaciones. Patrones básicos de arquitectura con IA. Consideraciones de latencia, costo, seguridad y rendimiento. Mantenibilidad de sistemas basados en IA. Validación de requisitos de calidad en sistemas con LLM.	- Ejemplo de aplicaciones con llamadas a LLM. - Práctica de desarrollo de caso de negocio con varias LLM.	- Diapositivas de clase - Casos en donde aplicar LLMs.	- Aprendizaje activo: Los estudiantes aprenden haciendo. - Aprendizaje colaborativo: Los estudiantes trabajan en equipo
11	RAG, MODELOS Y DATOS EN ARQUITECTURA CON IA Retrieval-Augmented Generation (RAG). Small Language Models (SLM). Integración con datos empresariales. Diseño de flujos de datos para sistemas con IA. Arquitectura orientada por datos. Integración con orquestador de flujos de trabajo.	- Desarrollo de ejemplo con chatbots y orquestadores de flujo. - Preparación y uso de RAG.	- Diapositivas de clase - Casos prácticos para el desarrollo de RAGs.	- Aprendizaje activo: Los estudiantes aprenden haciendo. - Aprendizaje colaborativo: Los estudiantes trabajan en equipo
12	AGENTIC IA Y ARQUITECTURAS VERIFICABLES Agentes de IA. Sistemas multiagente. Autonomía y coordinación de agentes. Riesgos y límites del software no determinista. Verificabilidad y validación empírica. Pruebas arquitectónicas en sistemas con IA.	- Conociendo las plataformas para Agentes IA. - Desarrollo de un Agente IA.	- Diapositivas de clase - Casos para aplicar agentes IA.	- Aprendizaje activo: Los estudiantes aprenden haciendo. - Aprendizaje colaborativo: Los estudiantes trabajan en equipo

13	PRIVACIDAD, SEGURIDAD Y DEVSECOPS Privacy by Design. DevSecOps. Seguridad integrada al ciclo de vida. Seguridad como código Trazabilidad, auditoría y cumplimiento. Riesgos de privacidad y uso de datos en soluciones con IA.	- Planificación del desarrollo de aplicación con DevSecOps. - Adopción de mecanismos de privacidad y protección de datos.	- Diapositivas de clase - Casos donde aplicar DevSecOps - Software: Github y Sonar y ZAP.	- Aprendizaje activo: Los estudiantes aprenden haciendo. - Aprendizaje colaborativo: Los estudiantes trabajan en equipo
14	SOSTENIBILIDAD Y GREEN SOFTWARE Sostenibilidad en arquitectura de software. Eficiencia energética. Uso responsable de infraestructura y nube. Decisiones arquitectónicas con impacto económico y ambiental.	Discusión de software de Sostenibilidad y Green Software	- Diapositivas de clase - Casos prácticos para desarrollo de software sostenible.	Aprendizaje activo: Los estudiantes aprenden haciendo.
15	ARQUITECTURA SOCIOTÉCNICA Y GOBERNANZA Arquitectura sociotécnica. Organización humana y decisiones de arquitectura. Gobernanza arquitectónica. Platform engineering e internal developer platforms. Sistemas mantenibles, sostenibles y verificables. Tendencias futuras en arquitectura de software.	Discusión de gobernanza de arquitectura	- Diapositivas de clase - Casos prácticos con Arquitectura Sociotécnica.	Aprendizaje colaborativo: Los estudiantes trabajan en equipo
16	EXPOSICIÓN FINAL DEL PROYECTO			

7. ESTRATEGIA DIDACTICA

Por parte del docente, se desarrollará la asignatura siguiendo los criterios formativo, interactivo, deductivo e inductivo, propiciando la participación del estudiante, fomentando la discusión crítica y orientada a la investigación. En este sentido se utilizará las técnicas de exposición de conceptos, con desarrollo de casos prácticos y resolución de problemáticas reales. Mediante el Aula Virtual se proporcionará diferentes recursos como son: diapositivas de clase, lecturas, videos, prácticas dirigidas con ejercicios que complementen los temas tratados.

Por parte del estudiante, participará a través de intervenciones en las sesiones de teoría y mediante el desarrollo de trabajos prácticos en clase. También realizará trabajos grupales de investigación y desarrollo de ejemplos que apliquen las tecnologías revisadas en clase. Los trabajos serán expuestos por grupos o individuales según sea el caso. El estudiante también llevará a cabo un proyecto para el curso, trabajado de forma grupal.

Estrategias:

- Aprendizaje activo: Los estudiantes aprenden haciendo.
- Aprendizaje colaborativo: Los estudiantes trabajan en equipo.
- Aprendizaje basado en problemas/proyectos: Los estudiantes resuelven problemas/proyectos reales.
- Metodología: Aula invertida, el profesor entrega material y se discute luego con lluvia de ideas
- Metodología Gamificación: puntaje por participación en clase y solución a problemas breves.

Recursos:

- Aula virtual: Moodle de la FISI
- Video conferencia: Microsoft Teams, Google Meet
- Pizarra en línea: OneNote
- Cuestionarios: Google Forms, Kahoot y Classmarker
- Otras herramientas: Miro, Figma, Jira, IcePanel, Visual Studio

Actividades:

- Exposición, lecturas, debates, programación y desarrollo del proyecto.

8. EVALUACION DEL APRENDIZAJE

La evaluación será como se indica a continuación:

Unidades de aprendizaje	Criterios y logros de aprendizaje	Procedimientos (Productos)	Instrumentos de evaluación	Pesos en porcentaje	
				Porcen.	Notas SUM
1, 2	Exposición parcial del proyecto	Presentación oral	Rúbrica del proyecto parcial	70%	N1
1, 2	Examen parcial	Prueba escrita individual	Prueba mixta	30%	
Total Nota 1				100%	
1, 2, 3	Desarrollo de casos de negocio con las arquitecturas y/o tendencias	Trabajos prácticos resuelto, demos	Rúbrica analítica (Se elimina la menor nota)	70%	N2
	Participación en clase	Intervenciones en clase y aportes en discusiones	Registro de participación	30%	
Total Nota 2				100%	
1, 2, 3	Exposición final del proyecto	Presentación y demo funcional	Rúbrica del proyecto final	80%	N3
2, 3	Artículo de investigación aplicado	Artículo individual tipo paper técnico	Rúbrica de evaluación del artículo	20%	
Total Nota 3				100%	
Promedio final = 0.3 * N1 + 0.4 * N2 + 0.3 * N3					

Rubrica de la evaluación final

Resultado de Aprendizaje	NO LOGRADO [0 – 2]	SUFICIENTE [3]	NOTABLE [4]	SOBRESALIENTE [5]
CT8.3 RA1: Diseñar soluciones de software que utilicen métodos y arquitecturas en tendencia, aplicando los conocimientos en diseño y arquitectura de software.	No diseña o diseña sin utilizar métodos o arquitecturas en tendencia.	Usa la arquitectura cebolla o limpia de manera básica	Usa la arquitectura cebolla o limpia de forma bien estructurada.	Usa la arquitectura hexagonal de forma estructurada.
CT8.3 RA2: Desarrollar aplicaciones de software con arquitecturas y tecnologías en nube, utilizando tendencias de arquitecturas de software.	No desarrolla utilizando tecnologías y arquitecturas en nube.	Desarrolla utilizando microservicios o sin servidor de forma correcta.	Desarrolla utilizando microservicios o sin servidor de forma correcta y utiliza CQRS o GraphQL.	Desarrolla utilizando microservicios o sin servidor de forma correcta y utilizaDDD, SAGA o basada en eventos.
CT8.3 RA3: Integrar aplicaciones de software utilizando tecnologías en tendencia, a partir del desarrollo de pruebas de concepto y demos.	No desarrollo pruebas de concepto o demos de tendencias.	Desarrolla pruebas de concepto de al menos dos tendencias.	Desarrolla demos básicos de al menos dos tendencias.	Desarrolla demos con casos de negocio avanzados de al menos dos tendencias.
CT8.3 RA4: Identificar y proponer la utilización de plataformas de software en tendencia para la solución de problemas reales y proyectos.	No propone ninguna solución para un problema real con una tecnología en tendencia.	Propone una solución para un problema real con una tecnología en tendencia.	Implementa, prueba y despliega una solución real con una tecnología en tendencia.	Implementa, prueba y despliega una solución real con dos tecnologías en tendencia.

9. REFERENCIAS BIBLIOGRAFICAS:

Especializada:

- Architecture Modernization: Socio-technical alignment of software, strategy, and structure, Nick Tune, Jean-Georges Perrin, 2024
- Software Architecture Patterns for Serverless Systems: Architecting for innovation with events, autonomous services, and micro frontends, John Gilbert, 2021
- Building Microservices: Designing Fine-Grained Systems, 2nd Edition, Sam Newman, 2021
- Software Architecture: The Hard Parts: Modern Trade-Off Analyses for Distributed Architectures, 1st Edition, Neal Ford, Mark Richards, Pramod Sadalage, Zhamak Dehghami, 2021
- API Design Patterns, JJ Geewax, 2021
- Fundamentals of Software Architecture: An Engineering Approach (1st Edition), Mark Richards, Neal Ford, 2020
- Software Architect's Handbook (1st Edition), Joseph Ingeno, 2018
- Arquitectura de Software. Conceptos y ciclo de desarrollo, Humberto Cervantes, Perla Velasco-Elizondo, Luis Castro, 2016
- Software Architecture in Practice (3rd Edition), Len Bass, Paul Clements, Rick Kazman, 2013
- Software Architect's Handbook (1st Edition), Joseph Ingeno, 2018
- Mark Richards. Software Architecture Patterns. 2015
- Clean Code: A Handbook of Agile Software Craftsmanship, 1st Edition, Robert C. Martin, 2008

- Chris Richardson. Microservices Patterns
- Luca Mezzalana. Building Micro-Frontends
- Adam Bellemare. Building Event-Driven Microservices

Virtual:

- <http://www.sei.cmu.edu/architecture/>
- <http://www.softwarearchitecturebook.com/>
- <https://c4model.com/>
- <https://www.infoq.com/articles/architecture-trends-2024/>
- <https://stratoflow.com/software-development-trends/>
- <https://cloud.google.com/blog/products/devops-sre/using-the-four-keys-to-measure-your-devops-performance>